

صل علي سيدنا محمد
ادعوا لي بالتوفيق ولكم بالمثل
افتكروا ابي و جدتي في دعائكم

Lecture 1

CMM levels

1. Level One : **Initial** – Work is performed **informally**. **worst**

A software development organization at this level is characterized by **AD HOC** عشوائي activities (organization is **not planned** in advance.).

2. Level Two : **Repeatable** – Work is **planned and tracked**.

This level of software development organization has a **basic and consistent project management processes** to **TRACK COST, SCHEDULE, AND FUNCTIONALITY**. The process is in place to **repeat the earlier successes on projects with similar applications**.

3. Level Three : **Defined** – Work is **well defined**.

At this level the **software process** for both **management and engineering activities** are **DEFINED AND DOCUMENTED**.

4. Level Four : **Managed** – Work is quantitatively controlled.

- **Software Quality management** – Management can effectively control the software development effort using precise measurements. At this level, organization set a quantitative quality goal for both software process and software maintenance.
- **Quantitative Process Management** – At this maturity level, The performance of processes is controlled using statistical and other quantitative techniques, and is quantitatively predictable.

5. Level Five : **Optimizing** – Work is Based Upon Continuous Improvement.

The key characteristic of this level is focusing on CONTINUOUSLY IMPROVING PROCESS performance.

Key features are:

- Process change management
- Technology change management
- Defect prevention

best

- Statistical Software Process Improvement (SSPI)

• **Error** Internal Discovery

- Some **flaw** أنثغة in a s/w engineering work product that is uncovered before the s/w is delivered to the **end-user**

• **Defect**

- A flaw that is uncovered after delivery to the **end-user**

External Discovery

S/W measurement can be categorized in two ways:

1. **Direct measures** of the s/w process (e.g., cost and effort applied) and product (e.g., lines of code (LOC) produced, etc.)
1. **Indirect measures** of the product (e.g., functionality, quality, complexity, etc.)

Requires normalization of both size- and function-oriented metrics

1-Size-Oriented Metrics

- **Lines of Code (LOC)** can be chosen as the normalization value
- Example of simple size-oriented metrics
 - **Errors** per KLOC (thousand lines of code)
 - **Defects** per KLOC
 - **\$** per KLOC
 - **Pages of documentation** per KLOC

3-Object-Oriented Metrics

- Enhanced version of Size-Oriented metrics, examples:

1. Number of **Scenario** scripts Use Case
1. Number of **key classes**
1. Number of **support classes**
1. Average number of **support classes** **per** **key class**
1. Number of **subsystems**

Defect Removal Efficiency (DRE)

- Can be used at both the project and process level

$$\text{DRE} = E / (E + D),$$

[E = Error, D = Defect]

- Try to achieve DRE that approaches **1**

1. Buffer Overflow

- Buffer **Overflow** occurs when **data** is written into a buffer (like array) past its end. (array or string)
- It may arise **due to** **faulty calculations** of the **write position**. Or continuous writing into a buffer **without checking the length**.
- Problems:
 - **Runtime Exception**
 - Or **runtime attack** (security problem)

2. SQL Injection

- **SQL Injection** is a technique for **injecting SQL commands** into **user input** such that these **commands** are **directly** executed by the database.
- This allows the **attacker to perform** malicious acts such **deleting tables, dropping databases, stealing data** and much more.

3.OS Command Injection

OS Command Injection arises when user-specified input is directly handed over to the operating system for execution by the application without proper vetting.

4. Integer Overflow or Wraparound

An **Integer Overflow** error occurs when you attempt to store a larger value into an integral type than will fit.

When this happens, the larger value is truncated and the operation ends up storing an unpredictable result.

7. Infinite loops

- Occurs when loop termination condition is **Never happening** (or takes too much time to happen)
- Example 1:
 - Language: c#
 - Source: <https://www.rosettacode.org/wiki/Loops/Infinite#C.23>
 - Code :

```
while (true)
{
    Console.WriteLine("SPAM");
}
```


- Codes: *in both codes the loop ending condition don't occur.*

ample 1:

```
short int i;  
for (i = 32765; i < 32768; i++)  
{  
    printf("%d\n", i);  
}
```

Example 2:

```
1 | int i = 1;  
2 |  
3 | while(i < 10)  
4 | {  
5 |     printf("%d\n", i);  
6 | }
```

- The following values are **always falsy**:
 - false
 - 0 (zero)
 - " or "" (empty string)
 - null
 - undefined
 - NaN
- Everything else is **truthy**. That includes:
 - '0' (a string containing a single zero)
 - 'false' (a string containing the text "false")
 - [] (an empty array)
 - {} (an empty object)
 - function(){} (an "empty" function)

12. Javascript == and === signs

In this example:

`0 == '0'`

the == operator considers the string to be integer, because it will make the two values the same type prior to comparing.

This can lead to all sorts of problems that are difficult to track down.

13. Confusing object comparison (== instead of .equals)

- The == operator compares two objects physically (their memory addresses) whereas the equals() method compares two objects semantically (their information).
- And in most of the cases, we should compare two objects meaningfully using the equals() method.
- Novice programmers often make a mistake by comparing two String objects like this:

صل علي سيدنا محمد
ادعوا لي بالتوفيق ولكم بالمثل
افتكروا ابي و جدتي في دعائكم

Lecture 2

What is Debugging?

Debugging is the process of finding and fixing errors or bugs in the source code of any software.

1. Syntax errors missing symbols, or wrong structure

A syntax error is a bug that occurs when a computer program has an incorrectly typed statement. It is the equivalent of a typo or spelling error in word processing. The program will not compile or run if syntax errors are present. The code editing software typically highlights this error.

2. Semantic errors

Semantic errors occur due to the improper use of programming statements.

3. Logic errors

Logic errors occur when programmers misrepresent the step-wise process or algorithm of a computer program. For example, the code may exit a loop too early or may have an incorrect if-then outcome. You can identify logic errors by stepping through the code for several different input/output scenarios.

4. Runtime errors

Runtime errors occur due to the computing environment in which the software code runs. Examples include insufficient memory space or stack overflow. You can resolve runtime errors by surrounding statements in try-catch blocks or logging the exception with an appropriate message.

What is Debugging?



- The process of **locating and fixing bugs** (errors) in program code
- Debugging tools (called **debuggers**)

Debugging Process

- The Debugging process includes:
 - **Spotting** an error
 - Finding the **lines** of code that caused the error
 - **Fixing** the error in the code
 - **Testing** to check if the error is gone and no new errors are introduced
- important
 - **Iterative and continuous process**
- Debuggers

Breakpoints



- Ability to **stop execution** based on certain criteria is key when debugging

Conditional Breakpoints



- Stop execution **only** when the set condition is **true**

"Custom" watch windows also possible

- Contain only variables that you choose to add

Locals watch window shows the local variables for the specific stack frame

- Displays: name of the variable, value and type

Autos lets the debugger decide which variables to show in the window

- Loosely based on the current and previous

Statement

Minification

Minification, also known as minimization, is the process of removing all unnecessary characters from JavaScript source code without altering its functionality.

This includes the:

1. removal of whitespace, comments, and semicolons,
2. along with the use of shorter variable names and functions.

Minification of JavaScript code results in compact file size.

Example: the following code contains 8 lines of code Before minification

```
1 // This function takes in name as a parameter
2 // and logs a string which greets that name
3 // using the information passed
4 function sayHi (name) {
5     console.log ("Hi" + name + ", nice to meet you." )
6 }
7
8 sayHi("Sam");
```

After minification: A single line of code

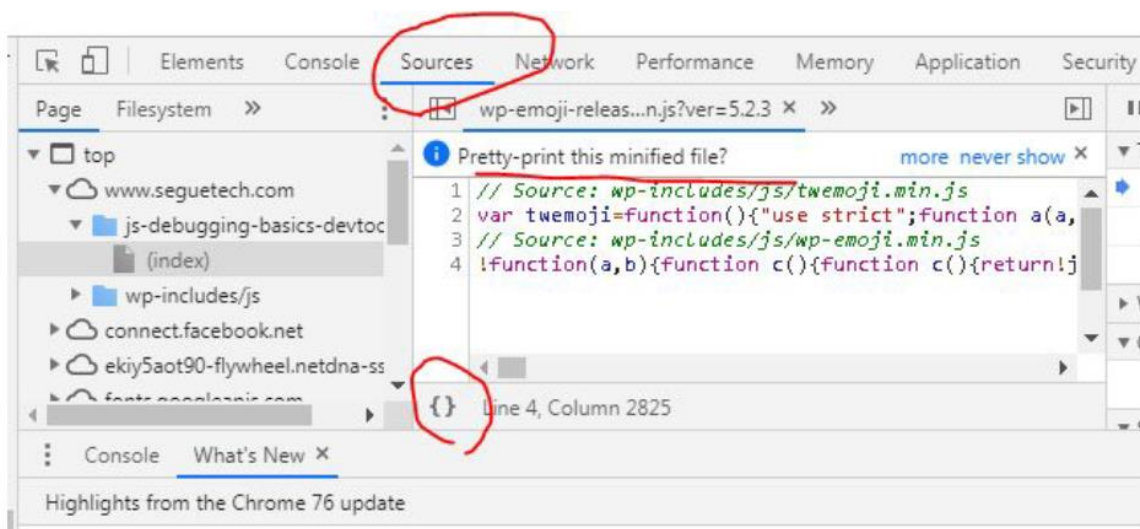
```
1 function sayHi(o){console.log ("Hi"+o+", nice to meet you.") }sayHi("Sam");
```

Result: Smaller, faster, harder-to-read code

Main benefits: Faster load, less bandwidth, quicker execution, fewer requests, better security.

Readable Code

- A lot of frameworks **minify their code** for **performance** which can be annoying for debugging purposes.
- Fortunately, DevTools comes with a useful feature **'Pretty Print', which will de-minify your code and make it readable.**
- Just go to your file and click the curly brackets icon **{ }** located at the bottom of the body panel.



WHAT IS GARBAGE COLLECTION?

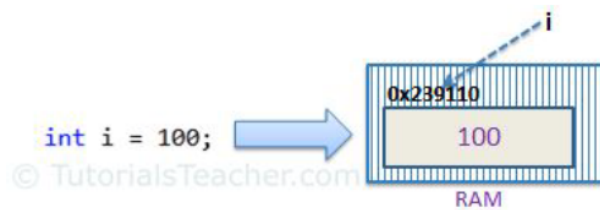
- **Memory Management technique.**
- Process of **automatic freeing objects** **No longer referenced** by the program.

WHY GARBAGE COLLECTION?

- Free **unreferenced** objects.
- Combat heap **fragmentation**.
- Relieves programmer from **manual freeing** the memory.
- Helps in ensuring program **integrity**.
- **Disadvantages**
 - Extra **Overhead**.
 - **Less control over scheduling** of CPU time.

Value type, and Reference type,

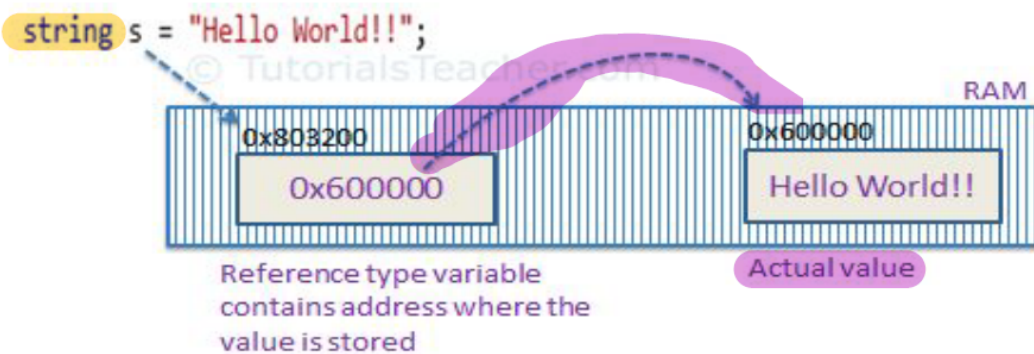
- **Value Type:** A data type is a value type if it holds a data value within its own memory space. It means variables of these data types directly contain their values.
- The following data types are all of value type: (c#)
 - bool
 - byte
 - char
 - decimal
 - double
 - enum
 - float
 - int
 - long
 - sbyte
 - short
 - struct
 - uint
 - ulong
 - ushort



Value type, and Reference type,

▣ Reference Type

- Unlike value types, a reference type doesn't store its value directly. Instead, it stores the address where the value is being stored. In other words, a reference type contains a pointer to another memory location that holds the data.
- Each Referenced Object contains 2 parts:
 - ▣ Object: the real data holder in heap



Multiple References for same object

- ▣ In reference type, it's possible that two or more references can point to the same object

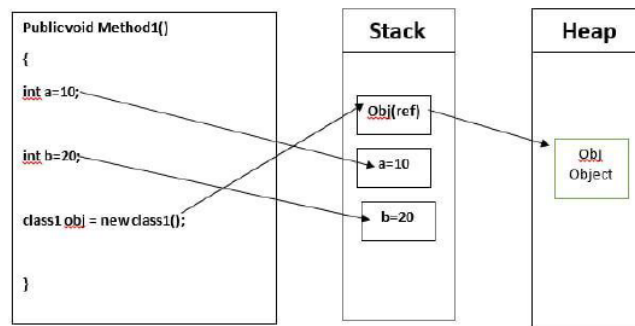
Stack, Heap

Stack Allocation :

- The allocation happens on **contiguous blocks** of memory.
- We call it stack memory allocation because the allocation happens in **function call stack**.
- The **size of memory to be allocated is known to compiler** and whenever a function is called, its variables get memory allocated on the stack.
- whenever the **function call is over**, the variable memory is **deallocated automatically**.

Heap Allocation :

- The memory is allocated during execution of instructions **written by programmers**.
- It is called heap because it is a **pile** **كومة** of memory space available to programmers to **allocated** and **de-allocate**.
- Allows objects with **dynamic size** in memory
- If a programmer does **not** handle this memory well, **memory leak** تسريب can happen in the program.



Unreachable objects

- An object is said to be **unreachable** if it **doesn't** contain **any reference** to it.
- Also note that objects which are part of **island of isolation** are also **unreachable**.

Island of Isolation

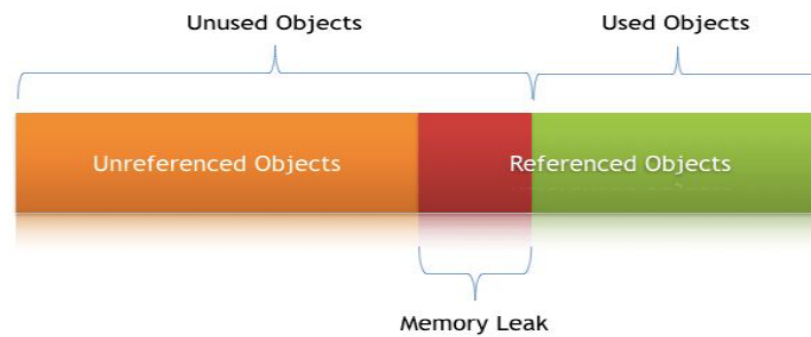
Basically, an island of isolation is a **group of objects** that **reference each other** but they are **not referenced by any active object** in the application.

LANGUAGES WITH AND WITHOUT GC

- Languages without GC
C, C++, D, Objective-C, Rust
 - Languages with GC
Java, Go, C#, PHP, Python, Scala, Haskell,
- Often the code quality is decreased to increase the performance

Memory Leak

- A Memory Leak is a situation when there are **objects present in the heap** that are **no longer used**, but the **garbage collector is unable to remove them from memory** and, thus they are **unnecessarily maintained**.
- A memory leak is bad because it blocks memory resources and degrades system performance over time.
- And if not dealt with, the application will eventually exhaust its resources, finally terminating with a fatal `java.lang.OutOfMemoryError`.



Profilers

- Wikipedia:
 - In software engineering, **profiling** ("program profiling", "software profiling") is a form of **dynamic program analysis** that **measures**, for example:
 - the **space** (memory)
 - **time complexity** of a program,
 - the **usage** of particular **instructions**,
 - or the **frequency** and **duration** of **function calls**.
 - Most commonly, profiling information **serves** to **aid program optimization**.
- Profilers are tools to **perform code Profiling**
 - Examples:
 - DotTrace (.net)
 - Netbeans Profiler, Visual VM (java)
 - Android studio Profiler
 - firefox/chrome browser profilers
 - Unity Profiler
 -

CPU profiler

- You can also monitor the **performance of each function or method in your code**

C# Optimization

The C# language is fast

- **A bit slower than C and C++**

Static fields are faster than instance fields

- **Instance methods are always slower than static**

Methods

- **Constants are fast**
- **StringBuilder can improve performance**

when appending strings

- Using char[] may be the fastest way to build up a string

- **Simple array T[] is always faster than**

List<T>

for-loops are faster than foreach loops

C# structs are slower

صل علي سيدنا محمد

ادعوا لي بالتوفيق ولكم بالمثل

افتكروا ابي و جدتي في دعائكم

Lecture 3

Code Visibility Levels During Testing

• There are 3 levels of Code Visibility:

1. **Black box testing** (also called functional testing):
is testing that **ignores the internal code** of a system or component and **focuses** solely on the **outputs** generated in response to **selected inputs** and **execution conditions**.
2. **White box testing** (also called structural testing and glass box testing):
is testing that takes into account the **full internal code** of a system or component.
3. **Gray Box testing**:
partial interaction of internal code of system/component.

Types of Test Techniques

1-Static Analysis Techniques –

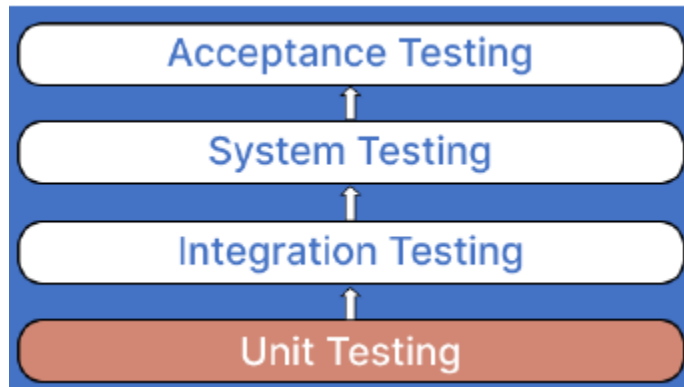
- Based **solely on** the (**manual or automated**) **examination** of project **documentation** of software **models** and **code**, and of other related information about requirements and design
- Generally yields valid results, but may be weak in precision
- فحص ملفات التوثيق والكود بدون تشغيل البرنامج

Testing is to check project documentation of software models and code without running the project in runtime.

2-Dynamic Test Techniques –

- Test the **software Execution** in order to find possible failures
- **Behavioral and performance** properties of the program are also observed
- Yields **more precise results**, but only holding for the examined execution أدق ولكن في الحالات المختبرة فقط

Dynamic testing requires running the code



detailed comparison between Unit and integration testing

Unit testing	Integration testing
Unit testing focuses on the individual modules of the application.	Integration testing focuses on the combined modules of the application.
It is usually the first level of testing but can be performed at any time.	It is performed after Unit testing and before System testing .
It can be performed by developers, testers, and QA engineers .	Only performed by testers .
It is a white-box testing technique.	It is a black-box testing technique.
It can be carried out without the completion of all the parts of the software.	Only be carried out after the completion of all the parts of the software.
It is easy to maintain, run and debug.	It's comparatively high maintenance and slower to run .
The issues are easy to find and can be instantly fixed.	The cost of fixing issues is higher and takes longer to resolve .
It focuses on module specification .	It focuses on interface specification .

Unit Testing

- Unit Testing checks that an individual program unit (subprogram, object class, package, module) behaves correctly.
- There is some debate about what constitutes a “unit”. Here some common definitions of a unit:
 - the smallest chunk that can be compiled by itself
 - a stand alone procedure or function
 - Class in OOP
 - something so small that it would be developed by a single person

Verification and Validation

- Software testing is part of a broader group of activities called verification and validation that are involved in software quality assurance
- **Verification (Are the algorithms coded correctly?)**
 - The set of activities that ensure that software correctly implements a specific function or algorithm
- **Validation (Does it meet user requirements?)**
 - The set of activities that ensure that the software that has been built is traceable to customer requirements

- test class
 - a set of input data that is identified for testing
- test case
 - specific data that is chosen for testing a test class.
- test Suite
 - a set of test cases that serve a particular testing goal
- exhaustive testing
 - test all logical paths in a software module

Stubs and Drivers

Stubs :

Stubs (similar to **children**) are dummy modules which are known as "**called programs**" which is used in integration testing (**top down** approach), used when sub programs are under construction.

Drivers:

Drivers (similar to **parents**) are also kind of dummy modules which are known as "**calling/caller programs**", which is used in **bottom up** integration testing, used when main programs are under construction. **Stub** means a Dummy model of a particular module.

5.1- Statement Coverage

- Statement coverage is a white box test design technique which involves execution of all the executable statements in the source code at least once.
- It is used to calculate and measure the number of statements in the source code which can be executed given the requirements.

$$\text{Statement Coverage} = \frac{\text{Number of executed statements}}{\text{Total number of statements}} \times 100$$

5.2. Decision Coverage

- Decision coverage reports the true or false outcomes of each Boolean expression.
- In this coverage, expressions can sometimes get complicated. Therefore, it is very hard to achieve 100% coverage.

$$\text{Decision Coverage} = \frac{\text{Number of Decision Outcomes Exercised}}{\text{Total Number of Decision Outcomes}}$$

5.3. Branch Coverage

- In the branch coverage, every outcome from a code module is tested. For example, if the outcomes are binary, you need to test both True and False outcomes.
- It helps you to ensure that every possible branch from each decision condition is executed at least a single time.
- By using Branch coverage method, you can also measure the fraction of independent code segments. It also helps you to find out which is sections of code don't have any branches.

$$\text{Branch Coverage} = \frac{\text{Number of Executed Branches}}{\text{Total Number of Branches}}$$

5.4. Condition Coverage

- Conditional coverage or expression coverage will reveal how the variables or **subexpressions** in the **conditional** statement are evaluated.
- In this coverage expressions with logical operands are only considered.
- For example, if an expression has Boolean operations like **AND, OR, XOR**, which indicated total possibilities.
- Conditional coverage offers **better sensitivity** to the control flow than decision coverage.
- **Condition coverage** does not give a guarantee about **full decision** coverage

$$\text{Condition Coverage} = \frac{\text{Number of Executed Operands}}{\text{Total Number of Operands}}$$

Smoke Testing

- **Build** the system that has new added features, and **run** it, make sure it runs **without building problems**.

Benefits of Smoke Testing

- **Integration risk is minimized**
 - Daily testing uncovers incompatibilities and show-stoppers early in the testing process, thereby reducing schedule impact
- The **quality of the end-product is improved**
 - Smoke testing is likely to uncover both functional errors and architectural and component-level design errors
- **Error diagnosis and correction are simplified**
 - Smoke testing will probably uncover errors in the **newest components** that were integrated
- **Progress is easier to assess**
 - As integration testing progresses, more software has been integrated and more has been demonstrated to work
 - Managers get a good indication that progress is being made

Alpha/Beta/Acceptance Testings

- **Alpha testing**
 - Conducted at the **developer's Environment** by **end users**
 - Software is used in a natural setting with **developers watching intently**
 - **Testing is conducted in a controlled environment**
- **Beta testing**
 - Conducted **at end-user Environment**
 - **Developer is generally not present**
 - It serves as a live application of the software in an environment that **cannot be controlled by the developer**
 - Main Purpose: The **end-user records all problems** that are encountered and reports these to the developers at regular intervals
 - **After beta testing is complete**, software engineers make **software modifications and prepare for release** of the software product to the entire customer base
- **Acceptance testing:**
 - Similar to beta testing, but the **main purpose** of it is to **get final user acceptance/approval of system to be delivered**

صل علي سيدنا محمد
ادعوا لي بالتوفيق ولكم بالمثل
افتكروا ابي و جدتي في دعائكم

Lecture 4

NUnit – Attributes

- [TestFixture] - This attribute marks a **class** that contains tests, and, optionally, setup or teardown methods
- [Test] - The Test attribute marks a **specific method** inside a class that has already been marked as a TestFixture, as a test method
- [TestFixtureSetUp] - Used to indicate a **setup method** that will **be run once; before all other tests**. This is the first method that is called **before the tests are started** مرة واحدة في البداية
- [TestFixtureTearDown] - Used to indicate a **tear down** **method that will be ran once; after all other tests have run**. This is the **last method that is called after all the tests have finished**. مرة واحدة في النهاية

NUnit – Attributes

- [SetUp] - Used to indicate a **setup method should be ran before each of the tests** قبل كل اختبار
- [TearDown] - Used to indicate a tear down method should be **ran after each of the tests are ran** بعد كل اختبار
- [ExpectedException(typeof(Exception))] –
 - When you **want** an exception to **be thrown**
 - Will only **pass** if exception type was thrown
- [Ignore("Not ready for primetime")] - Used when a test is **not ready**, or you don't want the test to be ran

Attribute	Purpose
[TestFixture]	Marks a class that contains tests
[Test]	Marks a test method
[SetUp]	Runs before each test
[TearDown]	Runs after each test
[TestFixtureSetUp]	Runs once before all tests
[TestFixtureTearDown]	Runs once after all tests
[ExpectedException]	Verifies a method throws a specific exception
[Ignore("message")]	Skips a test with a note

صل علي سيدنا محمد

ادعوا لي بالتوفيق ولكم بالمثل

افتكروا ابي و جدتي في دعائكم